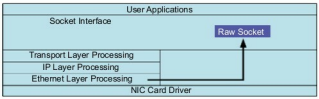

Figure 1: Graphical demonstration of a raw socket

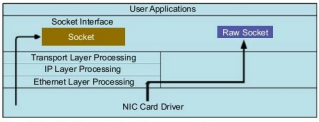

Figure 2: Graphical demonstration of how a raw socket works compared to other sockets

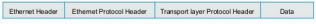

Figure 3: A generic representation of a network packet

Figure 4: Network packets for the Internet Protocol

available for Linux, like Wireshark. There is a command line sniffer called *tcpdump*, which is also a very good packet sniffer. And if we want to make our own packet sniffer, it can easily be done if we know the basics of C and networking.

A packet sniffer with a raw socket

To develop a packet sniffer, you first have to open a raw socket. Only processes with an effective user ID of 0 or the *CAP_NET_RAW* capability are allowed to open raw sockets. So, during the execution of the program, you have to be the root user.

Opening a raw socket

To open a socket, you have to know three things – the socket family, socket type and protocol. For a raw socket, the socket family is *AF_PACKET*, the socket type is *SOCK_RAW* and for the protocol, see the *if_ether.h* header file. To receive all packets, the macro is *ETH_P_ALL* and to receive IP packets, the macro is *ETH_P_IP* for the protocol field.

```
int sock_r;
sock_r=socket(AF_PACKET,SOCK_RAW,htons(ETH_P_ALL));
if(sock_r<0)
    {
      printf("error in socket\n");
      return -1;
    }
```

Reception of the network packet

After successfully opening a raw socket, it's time to receive network packets, for which you need to use the *recvfrom* API. We can also use the *recv* api. But *recvfrom* provides additional information.

```
unsigned char *buffer = (unsigned char *) malloc(65536); //to
receive data
memset(buffer,0,65536);
struct sockaddr saddr;
int saddr_len = sizeof (saddr);

       //Receive a network packet and copy in to buffer
buflen=recvfrom(sock_r,buffer,65536,0,&saddr,(socklen_t
*)&saddr_len);
if(buflen<0)
    {
      printf("error in reading recvfrom function\n");
      return -1;
    }
```

In *saddr*, the underlying protocol provides the source address of the packet.

Extracting the Ethernet header

Now that we have the network packets in our buffer, we will get information about the Ethernet header. The Ethernet header contains the physical address of the source and destination, or the MAC address and protocol of the receiving packet. The *if_ether.h* header contains the structure of the Ethernet header (see Figure 5).

Now, we can easily access these fields:

```
struct ethhdr *eth = (struct ethhdr *)(buffer);
printf("\nEthernet Header\n");
printf("\t|-Source Address : %.2X-%.2X-%.2X-%.2X-%.2X-%.2X\
n",eth->h_source[0],eth->h_source[1],eth->h_source[2],eth-
>h_source[3],eth->h_source[4],eth->h_source[5]);
printf("\t|-Destination Address  : %.2X-%.2X-%.2X-%.2X-%.2X-
%.2X\n",eth->h_dest[0],eth->h_dest[1],eth->h_dest[2],eth-
>h_dest[3],eth->h_dest[4],eth->h_dest[5]);
printf("\t|-Protocol      : %d\n",eth->h_proto);
```

h_proto gives information about the next layer. If you get *0x800 (ETH_P_IP)*, it means that the next header is the IP header. Later, we will consider the next header as the IP header.

Note 1: The physical address is 6 bytes.
Note 2: We can also direct the output to a file for better understanding.

```
fprintf(log_txt,"\t|-Source Address  : %.2X-%.2X-%.2X-
%.2X-%.2X-%.2X\n",eth->h_source[0],eth->h_source[1],eth-
>h_source[2],eth->h_source[3],eth->h_source[4],eth->h_
source[5]);
```